软件需求分析与设计

第一章框架梳理

仅供学习参考

目 录

一. 软件开发的本质 1

1、70%的项目是失败的（2005 年的数据）：质量差，超期，超预算，需求变更 1

2、 软件本身是创造性开发行为的产品 1

3、 软件开发不可变的事实是（无能为力解决，只能承认接受） 1

(1) complexity 复杂性 1

(2) conformity 一致性：软件与平台，系统一致 1

(3) Changeability（需求）可变性 1

(4) invisibility（代码）不可见性 1

4、 可变的意外困难，不会增加软件开发的复杂性 1

(1) 社会问题 1

(2) 提高软件的可适应性=可理解+可维护+可扩展 1

5、 软件开发的意外事件 1

(1) Stakeholder 利益相关者：对系统产生影响或者被系统所影响的人，是在软件项目中存在利害关系的人 1

(2) Process 过程：在软件开发和维护过程中定义的活动和组织程序 1

(3) Modeling 建模：需要开发人员在不同阶段进行沟通（Language——UML）和文档化（Tools——CASE） 2

6、 集成方法 2

(1) 面向信息的集成 2

(2) 面向门户的集成 2

(3) 面向接口的集成 2

(4) 面向过程的集成 2

二、系统计划 3

1、 系统计划的不同方法 3

(1) SWOT——Strengths，Weakness，Opportunities，Threats 3

(2) VCM（Value Chain Model）——价值链模型 3

(3) BPR（business process reengineering）——业务过程重组 3

(4) ISA（Information Systems Architecture）——信息系统体系结构 3

三、 系统的三级管理 5

1. 操作级 Operational 5

(1) 由底层操作员，做一些日常工作。如发工资，收银员 5

仅供学习参考

(2) 关键概念：数据——原始事实.....5
(3) 运用工具：事务处理系统——联机事务处理（On-Line Transaction Processing ， OLTP）系统.....5
2. 战术级 Tactical.....5
(1) 做短期目标，进行信息资源分配，对已处理数据分析归纳。如预算分析，工资预测，调度，客服.....5
(2) 关键概念：信息——有用的数据，增值的事实.....5
(3) 运用工具：分析处理系统——联机分析处理（On-Line Analytical Processing ， OLAP）系统.....5
3. 策略级 Strategic.....5
(1) 行政和高层管理级，制定系统产品长期计划，更强调预测性，从已知得到未知。如市场预测.....5
(2) 关键概念：知识——对信息的理解和有效使用.....5
(3) 运用工具：知识处理（Knowledge Processing）系统.....5
四、系统开发周期——核心内容.....6
1. 开发方法.....6
(1) 传统软件开发方法是结构化方法，以过程为中心.....6
(2) 现代软件开发方法是面向对象方法，基本特点是交互性.....6
2. 生命周期——测试贯穿整个周期.....6
(1) 业务（需求）分析——整个生命周期最重要的阶段，没有模块的概念.....6
(2) 系统设计.....7
(3) 实现（安装 + 编码 + 调试）.....7
(4) 集成和部署.....7
(5) 运行和维护.....7
3. 跨越生命周期的活动.....8
(1) 项目计划 Project Planing.....8
(2) 度量 Metrics.....8
(3) 测试 Testing.....8
五. 开发模型和方法——关于软件的生产方式.....9
1. 螺旋模型（Spiral Model）.....9
(1) 最本质，风险驱动.....9
2. IBM Rational 统一过程（RUP）.....9
(1) 软件开发过程平台.....9
(2) RUP 在二维关系中组织项目.....9
3. 模型驱动的开发（MDA）.....9
(1) 基于 UML，利用模板。.....9

仅供学习参考

(2) 旨在得到与平台无关的模型，包括系统状态和行为的完全规格说明。 9
(3) 在下一步骤中，MDA 提供了工具和技术来创建特定平台的模型。 9
4. 敏捷开发模型（Agile Software Development） 9
(1) 越快越好，尽快给出成果让用户评价。 9
(2) 测试驱动的开发——编写程序来通过验收测试。 9
(3) 策划（Intentional）编程——在开始编码前，具体指定验收测试中程序想要获得的能力和机会。 9
5. 面向方面的开发（Aspect-Oriented Programming，AOP） 9
(1) 适用于功能上有区别但是大部分相同的软件。 9
(2) 面向方面的开发正是应用迭代和增量过程来生产适应性软件的另一种方式。 9

仅供学习参考

一. 软件开发的本质

1、70%的项目是失败的（2005 年的数据）：质量差，超期，超预算，需求变更

2、软件本身是创造性开发行为的产品

3、软件开发不可变的事实是（无能为力解决，只能承认接受）

(1) complexity 复杂性

(2) conformity 一致性：软件与平台，系统一致

(3) Changeability（需求）可变性

(4) invisibility（代码）不可见性

4、可变的意外困难，不会增加软件开发的复杂性

(1) 社会问题

(2) 提高软件的**可适应性=可理解+可维护+可扩展**

5、软件开发的意外事件

(1) Stakeholder 利益相关者：对系统产生影响或者被系统所影响的人，是在软件项目中存在利害关系的人

① 客户

② 开发者

(2) Process 过程：在软件开发和维护过程中定义的活动和组织程序

① 迭代（迭代完发布新版本，反复求精）增量（一次迭代的功能增加或减少，逐块构建，非功能性软件质量的改进）模型，需要提前做好计划和控制，和预定义的设计框架保持一致

1) Spiral Model（螺旋模型）：包含了 RUP，MDA，敏捷开发模型

2) RUP（Rational Unit Process）：提供文本，概念，开发思路

3) MDA（模型驱动的体系结构）

4) Agile（敏捷）开发过程

5) Aspect-oriented 面向方面的软件开发

② 能力与成熟度模型（Capability Maturity Model ,CMM）用于过程评估和改进的流行方法。

1) 第一级：走到哪算到哪，没有特定的规则，大部分都在第一级

2) 第二级：可重复级，靠经验每一次用相同的方法，东软

3) 第三级：定义级，已经定义好了规则，不到整个软件企业总量的 10%

4) 第四级：管理级，对整个每一步的过程增加了监控，亚马逊

5) 第五级：优化级，对整个过程或标准优化的策略，基本是军方企业

③ ISO 9000 质量标准系列——工业生产的通行标准

1) 如果过程是正确的，那么结果一定是正确的，即关注产品质量问题

2) 强调必须完成什么，没强调必须怎么执行

仅供学习参考

④ IT 基础架构库（ITIL）

1) 高效的利用 4P：People, Processes, Products, Partners

2) 致力于方案交付和管理的操作方便

3) 持续的服务改进方案(Continuous Service Improvement Programme, CSIP)：用来实现解决方案管理的 ITIL 方法。该方案以实现高水平业务目标的决心为起点，接着检查是否达到里程碑，并通过巩固已达到的改进和持续任务循环而保持发展的势头。

⑤ COBIT 框架（控制目标信息和相关技术），了解

1) 是一个产品标准：侧重组织需要做什么，而非如何去做

2) 将相关的 IT 工作组织到 4 个领域

a. 规划与组织

b. 获取与实现

c. 交付与支持

d. 监控

(3) Modeling 建模：需要开发人员在不同阶段进行沟通（Language——UML）和文档化（Tools——CASE）

① UML 独立于任何软件开发过程，也独立于实现技术

1) 要求：只要面向对象开发方法就行

2) 缺陷：必须采用面向对象的方法来生产软件

② UML 模型

1) 状态模型 State Models：类图。描述静态数据结构

2) 行为模型 behavior models，用例图，时序图，活动图。描述对象协作

3) 状态转换模型 state change models，状态图。描述随着时间推移，系统所允许的状态

6、集成方法

(1) 面向信息的集成

① 面向信息关注信息的实时交换，这是在数据库应用程序接口 API 层次的集成

(2) 面向门户的集成

① 是特殊的面向信息的集成

② 将来自多个软件系统的信息具体化到一个共同的用户界面

③ 需要来自后台系统的具体信息进行人为干预

(3) 面向接口的集成

① 将应用接口（通过接口抽象定义的服务）连接在一起

② 接口显示了一个应用系统向其他应用系统所提供的有益服务。

(4) 面向过程的集成

① 将应用系统连接在一起，在现有应用系统的已有过程集和数据集的顶部定义一个新的过程层。

仅供学习参考

二、系统计划

可以通过各种策略规划、业务建模、业务过程重组、策略调整、信息资源管理或诸如此类的过程决定业务策略。以上方法目的是为业务确定长远洞察力，然后优先考虑能够通过使用信息技术而得以解决的业务问题。

所有系统规划的方法有一个共同点——先关注效果再关注效率，即先有效 effectiveness，再高效 efficiency

1、系统计划的不同方法

(1) SWOT——Strengths，Weakness，Opportunities，Threats

① 内涵：通过调整组织优势、劣势、机会和威胁的方式来进行 IS 开发项目的识别、分类、排序和选择。这是一个从确定组织使命开始的、自顶向下的方法。

② 使命陈述捕获一个组织的独特性质，并详细说明它未来的愿景，良好的使命陈述重点在于客户的需要。

③ 对内部的企业优势和劣势的识别是成功业务规划的必要非充分条件

④ 对可利用的外部机会和可避免的外部威胁的认识是决定组织目的和目标的基础

⑤ 实质是看清自己，内因：优势劣势， 外因：机遇挑战

⑥ 得到 SWOT 矩阵，基于每一个问题细化目标，得到策略

(2) VCM（Value Chain Model）——价值链模型

① 内涵：分析组织中完整的活动链（从原材料到销售及运送给客户的最终产品）来评估竞争优势

② 目的：理解哪种价值链配置将产生最大竞争优势

③ 组织的职能：

1) 基本活动（Primary Activity）——Create or add value to a final product，如教学培养计划

a. 内部物流——接受对产品或服务的投入

b. 操作——使用投入来创造产品或服务

c. 外部物流——将产品和服务卖给买家

d. 销售和市场——引导买家购买产品或服务

e. 服务——维护或提高产品或服务的价值

2) 支持活动（Support Activity）——Essential but they DON'T enrich the product，如学生食堂

3) 基于不同业务场景来区分主要次要活动

(3) BPR（business process reengineering）——业务过程重组

① 前提：当今组织必须彻底改造自己，并丢弃现在使用的功能分解、分层结构和操作原则

② 主要目的：在组织中从根本上重新设计业务过程。必须对业务过程进行识别、流程化和改进。在工作流图中对过程文档化，并经历工作流分析。

③ 主要障碍：需要向传统的总想管理结构中嵌入一个横向过程。

(4) ISA（Information Systems Architecture）——信息系统体系结构

仅供学习参考

① 自底向上的方法，它为能够适应各种业务策略的 IS 解决方案提供一种中立的系统结构框架。

② ISA 框架：分为 5 行 6 列，5 种人 6 个阶段

1) 五种人

a. 规划者

b. 所有者 Owner

c. 设计者

d. 建造者

e. 承包者 Subcontractor

③ 6 列：表现每个参与者所从事的 6 种不同的描述或体系结构模型。

④ 规定软件过程中每一个阶段的成果由谁来完成的

仅供学习参考

三、系统的三级管理

1. 操作级 Operational

(1) 由底层操作员，做一些日常工作。如发工资，收银员

(2) 关键概念：数据——原始事实

(3) 运用工具：事务处理系统——联机事务处理（On-Line Transaction Processing ， OLTP）系统

① 事务：工作的一个逻辑单元，完成某一特定的业务任务，并在任务完成后保证数据库的完整性

② 关联技术：数据库技术

③ OLTP 中事务管理的两大主要功能

1) 并发控制（Concurrency Control）

2) 从软硬件故障中恢复（Recovery from failures）

2. 战术级 Tactical

(1) 做短期目标，进行信息资源分配，对已处理数据分析归纳。如预算分析，工资预测，调度，客服

(2) 关键概念：信息——有用的数据，增值的事实

(3) 运用工具：分析处理系统——联机分析处理（On-Line Analytical Processing ， OLAP）系统

① 关联技术：数据仓库（Data Warehouse）技术。

1) 创建方式通常是在一个或多个事务数据库中提取的数据增量拷贝。

2) 总是增加新数据，不移除历史数据。

3) 独特特征：对数据的汇总和封装，目的是给源系统提供的数据增加价值

② 其他相关技术

1) 数据集市（Data Mart）——主要保存被汇总的历史数据

2) Data Web-house

3. 策略级 Strategic

(1) 行政和高层管理级，制定系统产品长期计划，更强调预测性，从已知得到未知。如市场预测

(2) 关键概念：知识——对信息的理解和有效使用

(3) 运用工具：知识处理（Knowledge Processing）系统

① 关联技术：数据挖掘（Data Mining）——主要目的：

1) 关联（路径分析）Association——在数据中发现一个事件导致另一个相关事件发生的模式

2) 分类 Classification——发现某事实是否落入预定、感兴趣的类别中

3) 聚类 Clustering——与分类相似，但种类未知，它们由聚类方法发现

② 用于数据挖掘的数据主要来源于数据仓库，而不是操作型数据库

仅供学习参考

四、系统开发周期——核心内容

1．开发方法

(1) 传统软件开发方法是结构化方法，以过程为中心

① 系统在**功能分解**活动中被分解为可管理的单元，并将系统有层次的划分为由数据流连接的业务过程

② 2004 年前基本上是结构化

③ 基本工具——UML 不是系统开发的结构化方法的建模技术

1) 数据流图 DFD（Data Flow Diagrams），过程建模，结构化方法的开发驱动力

2) 实体关系模型 ERD（Entity Relationship Diagrams），数据建模

④ 不适于现代软件开发原因

1) 结构化方法是顺序，转换的方法，而不是迭代增量的方式，现代软件要求迭代增量

2) 写好的软件不可变无弹性，即很难对业务功能集进行扩展和沿伸，重用性差

3) 基本上所有的软件从零开始写，不支持已存在构件的复用

(2) 现代软件开发方法是面向对象方法，基本特点是交互性

① 事件驱动，软件执行顺序之前不知道

② 用户控制着程序的执行

③ 软件服务响应随机，是不可预测的

④ 以数据为中心，在分析阶段不需要定义类的操作，只需要定义类的属性，遵循迭代增量的过程

⑤ 存在问题：

1) 理解较困难，结构较复杂，分析阶段在一个相当高的抽象层面进行

2) 数据库大多数是关系型数据库，怎么把对象存在数据库里

3) 项目管理更加困难

2．生命周期——测试贯穿整个周期

(1) 业务（需求）分析——整个生命周期最重要的阶段，没有模块的概念

① 主要目的：获取需求 + 建模

② 两个阶段：

1) 获取和确定业务需求，需要业务分析人员（可以没有软件开发背景，需要与人良好沟通）

2) 将业务需求逻辑转换为系统逻辑，对需求进行详细说明或建模，需要系统分析员

③ 需求确定 determination

1) 服务陈述（Service Statement）

a. 系统的业务规则（Business rule），如两周发一次工资

b. 系统计算过程（Computation），如怎么发这个工资，根据前两周销售业绩发工资

2) 约束陈述（Constraint Statement）

a. 使用约束——使用时的限定，如选课时每个人只能查看自己的账号信息

b. 开发约束——与系统开发过程所需要软硬件平台的约束

④ 需求获取 specification 规格说明

1) 建模方法和工具：

a. CASE 工具

b. 需求文档

2) 最重要的两个规格说明技术

a. 类图（描述系统的对象及对象间的关系，静态数据结构），需求分析阶段得不到完整类图

b. 用例图

⑤ 在理想状态下，需求分析阶段可以与软硬件平台无关，且非功能性需求在用例图和类图中无法描述

(2) 系统设计

① 体系结构设计 Architectural Design——根据模块（构件）进行的系统描述

1) 选择一个解决方案的策略，比如用哪个版本的数据库

2) 系统模块的划分

3) 客户机服务器模型经常被扩展为三层体系结构，中间层是逻辑层，实现为动态链接库。

4) 一个良好的体系结构设计会产生可适应（支持）的系统——可理解、可维护、可升级

② 详细设计 Detailed Design——软件构件内部运行的描述

1) 考虑模块内部的工作过程，如确定需要哪些数据结构（类图描述），算法（伪代码描述）

(3) 实现（安装 + 编码 + 调试）

① 涉及所购买软件的安装和客户定制软件的编码

② 成果：可运行的程序代码

③ 客户端的实现和服务器端数据库的实现

(4) 集成和部署

① 定义：软件以较小的模块开发，在为客户用于生产而部署之前，这些构件需要与已经可操作的模块组装且集成在一起（模块集成+应用程序集成）

② 对于大型系统来说，模块集成会比任何早期生命周期阶段（包括实现阶段）花费更多时间和精力

③ 面向对象的系统必须为集成和部署而设计

(5) 运行和维护

① 运行：把系统交给用户进行实际使用——标志着从现有业务解决方案到新方案的转换

② 维护

1) 内务处理——必要的日常维护任务，以保持系统的可访问性和可操作性。

2) 适应性维护——对系统运行的监控、审核、功能调整，以适应系统性能、满足变化的环境。

3) 完善性维护——对系统的重新设计和修改，适应新需求，让系统越变越好。

③ 淘汰（Phase Out）：很难再从软件的可用性中获益的系统将被淘汰。

3．跨越生命周期的活动

(1) 项目计划 Project Planing

① 最终软件产品交付什么，花费，时间，风险，里程碑，资源分配，也包括开发方法、过程、工具、标准和团队组织的选择。

② 典型的约束是时间和费用——每个项目都有清晰的期限和紧张的预算。

(2) 度量 Metrics

① 测量开发时间、工作量等，常常在软件质量和复杂性的范围内讨论度量。

② 用于测量正确性、有效性，完整性，可用性，可维护性，灵活性，可测试性等测量因素。

③ 度量还可以在生命周期的不同阶段测量开发模型，评估过程的效果并改善工作质量。

(3) 测试 Testing

① 跨越整个软件生命周期的一项活动，系统涉及一点变化就需要测试。

1) 基于执行的测试

a. 规格说明测试——黑盒测试：检查输出正确性和需求完成度

b. 代码测试——白盒测试：检查程序逻辑错误

仅供学习参考

五. 开发模型和方法——关于软件的生产方式

1．螺旋模型（Spiral Model）

(1) 最本质，风险驱动

2．IBM Rational 统一过程（RUP）

(1) 软件开发过程平台

(2) RUP 在二维关系中组织项目

① 横向维度——项目迭代的连续阶段

1) 初始

2) 细化

3) 构造

4) 转换

② 纵向维度——软件开发领域

1) 业务建模

2) 需求

3) 分析和设计

4) 实现

5) 测试

6) 配置及变更管理的部署和支持活动

7) 项目管理和环境

③ 像螺旋模型一样，RUP 强调迭代开发和早期的、持续的风险分析的重要性。

④ RUP 具有普遍适用性，但是也使用了 IBM Rational 软件开发工具为团队提供特别指导。

3．模型驱动的开发（MDA）

(1) 基于 UML，利用模板。

(2) 旨在得到与平台无关的模型，包括系统状态和行为的完全规格说明。

(3) 在下一步骤中，MDA 提供了工具和技术来创建特定平台的模型。

4．敏捷开发模型（Agile Software Development）

(1) 越快越好，尽快给出成果让用户评价。

(2) 测试驱动的开发——编写程序来通过验收测试。

(3) 策划（Intentional）编程——在开始编码前，具体指定验收测试中程序想要获得的能力和机会。

5．面向方面的开发（Aspect-Oriented Programming，AOP）

(1) 适用于功能上有区别但是大部分相同的软件。

(2) 面向方面的开发正是应用迭代和增量过程来生产适应性软件的另一种方式。

仅供学习参考